


# Data Mining and ML for Fault Detection in SECOM Dataset

**Student Name: Chetan Babu Mahendiran**

**For the module STAT9013 - Data Mining and Stat Modelling part of the**

**Masters in Science in Data Science and Analytics, Department of Mathematics**

# Table of Contents

**ABSTRACT** ..... **3**

**1. INTRODUCTION** ..... **4**

**2. RELATED WORK** ..... **5**

**3. METHODOLOGY** ..... **6**

3.1 Dataset ..... 6
3.2 Preprocessing ..... 6
3.3 Exploratory Data Analysis ..... 8
3.4 Outlier Analysis ..... 10
3.5 Dimensionality Reduction (PCA) ..... 11
3.6 Clustering ..... 13
3.7 Classification ..... 15
3.8 SMOTE ..... 17
3.9 Model Evaluation ..... 17

**4. EXPERIMENTS AND RESULTS** ..... **19**

4.1 Preprocessing Outcomes ..... 19
4.2 EDA Findings ..... 19
4.3 Outlier Detection ..... 19
4.4 PCA Results ..... 20
4.5 Clustering Results ..... 20
4.6 Classification Results ..... 21

**5. DISCUSSION** ..... **22**

**6. CONCLUSION** ..... **25**

**7. REFERENCES** ..... **26**

**APPENDIX** ..... **27**

Data Understanding ..... 28
EDA ..... 31
Outlier ..... 32
PCA ..... 34
K - MEANS ..... 35
Classifiers ..... 37
KNN ..... 37
Decision Tree ..... 38
SVM Classifier ..... 39
SMOTE ..... 40
Model Evaluation ..... 41

# Abstract

This report provides a detailed survey of an extensive data mining study on the SECOM semiconductor manufacturing dataset containing 1,567 samples and 591 sensor features collected from actual semiconductor manufacturing process. This dataset exhibits severe class imbalance, 93.4% of samples are labelled passing and only 6.6% failing alongside a high quantity of missing sensor readings. This work aims at constructing and evaluating an entire pipeline, from raw data to valuable predictive models, including preprocessing, exploratory data analysis, outlier detection dimensionality reduction with PCA, unsupervised K-Means clustering and supervised k-NN-based classification Decision Tree , SVM. Class imbalance was addressed before classification using the Synthetic Minority Oversampling Technique (SMOTE). Results show that Decision Tree with SMOTE gave the best balance between precision and recall, and results of clustering analysis using K-means indicate that unsupervised methods on this dataset are not sufficient to separate passing and failing products.

# 1. Introduction

One of the biggest industrial processes in the world, semiconductor manufacturing is also one of the most complex and quality-sensitive. Even minor variations in process conditions can cause a product to fail quality checks and fail, leading to a significant financial loss. So because of this, detecting these failures early, and understanding which sensor signals predict them, is a very useful problem for data mining and machine learning.

Exactly this challenge is what the publicly available SECOM dataset collected from a semiconductor manufacturing process and provided by UCI Machine Learning Repository resembles. It includes data for hundreds of sensors measured during the production of single units, each accompanied by a binary label that indicates whether some quality test on that unit passed or failed. It is worth mentioning that the dataset is messy in a realistic manner with many missing values, thousands of features per sample , and an extreme imbalance of positive class samples.

The goal of this project is to apply a full data mining workflow to this dataset. Starting from raw data, we clean, explore, reduce, cluster, and classify it, evaluating each step along the way. The key questions we aim to answer are:

- Which preprocessing steps are necessary to make this data usable
- How many principal components are needed to capture most of the variance
- Whether unsupervised clustering can reveal structure related to the pass/fail labels
- Which supervised classifier performs best at identifying failing units.

## 2. Related Work

Since the release of the SECOM Dataset, multiple approaches have been proposed to study the data in industrial fault detection research. The problem was formulated as fault detection for high-dimensional noisy industrial environments by McCann and Johnston [4], who emphasized that extreme class imbalance and the high feature-to-sample ratio are the two most challenging aspects that any method must address.

Oversampling methods such as SMOTE, proposed by Chawla et al. [1], have been widely applied for imbalanced classification and have demonstrated consistent improvements in minority-class recall while avoiding test-set data leakage. Studies have shown that standard classifiers trained on the SECOM dataset without imbalance handling tend to predict almost exclusively the majority class, resulting in very high accuracy but near-zero recall for failing units [1].

The use of Principal Component Analysis (PCA) for reducing the dimensionality of high dimensional sensor data is a classical and well-established approach. The theoretical foundation provided by Jolliffe [3] has been validated extensively in industrial applications. In most cases, retaining 95% of the explained variance provides an effective balance between information preservation and computational efficiency.

Support Vector Machine (SVM) models with RBF kernels have shown strong performance on high-dimensional datasets [2], making them a suitable choice for the classification task in this work after PCA-based feature reduction. Decision Trees provide interpretability but are generally less robust to noisy data. Meanwhile, k-Nearest Neighbors (k-NN) serves as a simple yet effective baseline model; however, it is strongly affected by the curse of dimensionality, making evaluation in the PCA-reduced feature space particularly relevant.

# 3. Methodology

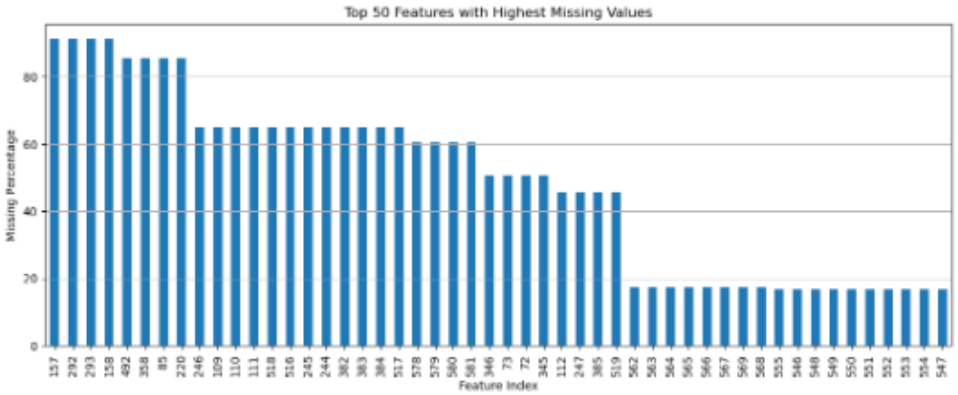
## 3.1 Dataset

The SECOM data is two files secom. data, which covers 1,567 rows and 590 columns of sensor measurements, and secom_labels. Data corresponding label based on class type (-1 = pass, 1 = fail), along with a timestamp. In this analysis, we did not make use of the timestamp column.

## 3.2 Preprocessing

Preprocessing followed a four-step pipeline.

- Features with more than 50 % of their values are missing were eliminated. This cutoff was chosen based on best practice, since features with >50% missing data are unlikely to hold a good signal and imputing these will likely induce more noise than information.




```
Features with more than 50% missing values:
28
```

- We used median imputation for all remaining missing values using SimpleImputer from scikit-learn. The median was selected instead of the mean because sensor data is often highly skewed due to a few extreme readings and medians are more robust.
- Constant columns were removed because they do not contain any discriminative information.
- StandardScaler were used to standardise all features, scaling each feature with zero mean and unit variance. This is required for PCA and Euclidean distance-based classifiers like k-NN and SVM that are sensitive to feature scale.

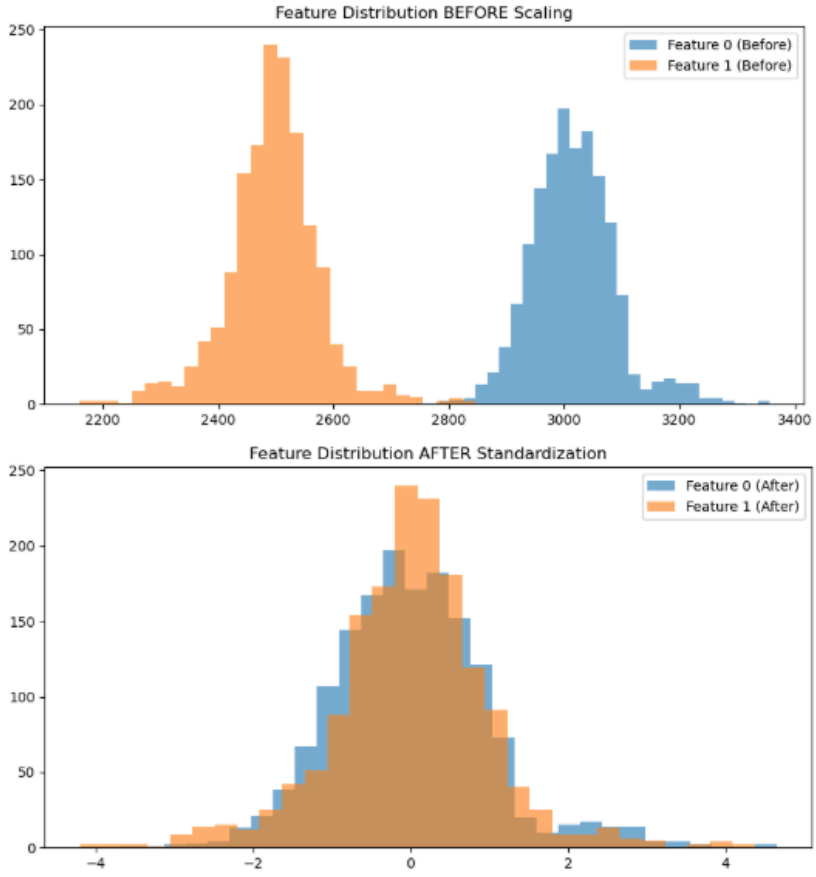
BEFORE SCALING (first 5 features):

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| **count** | 1567.00 | 1567.00 | 1567.00 | 1567.00 | 1567.00 |
| **mean** | 3014.44 | 2495.87 | 2200.55 | 1395.38 | 4.17 |
| **std** | 73.48 | 80.23 | 29.38 | 439.84 | 56.10 |
| **min** | 2743.24 | 2158.75 | 2060.66 | 0.00 | 0.68 |
| **25%** | 2966.66 | 2452.89 | 2181.10 | 1083.89 | 1.02 |
| **50%** | 3011.49 | 2499.40 | 2201.07 | 1285.21 | 1.32 |
| **75%** | 3056.54 | 2538.74 | 2218.06 | 1590.17 | 1.52 |
| **max** | 3356.35 | 2846.44 | 2315.27 | 3715.04 | 1114.54 |

AFTER SCALING (first 5 features):

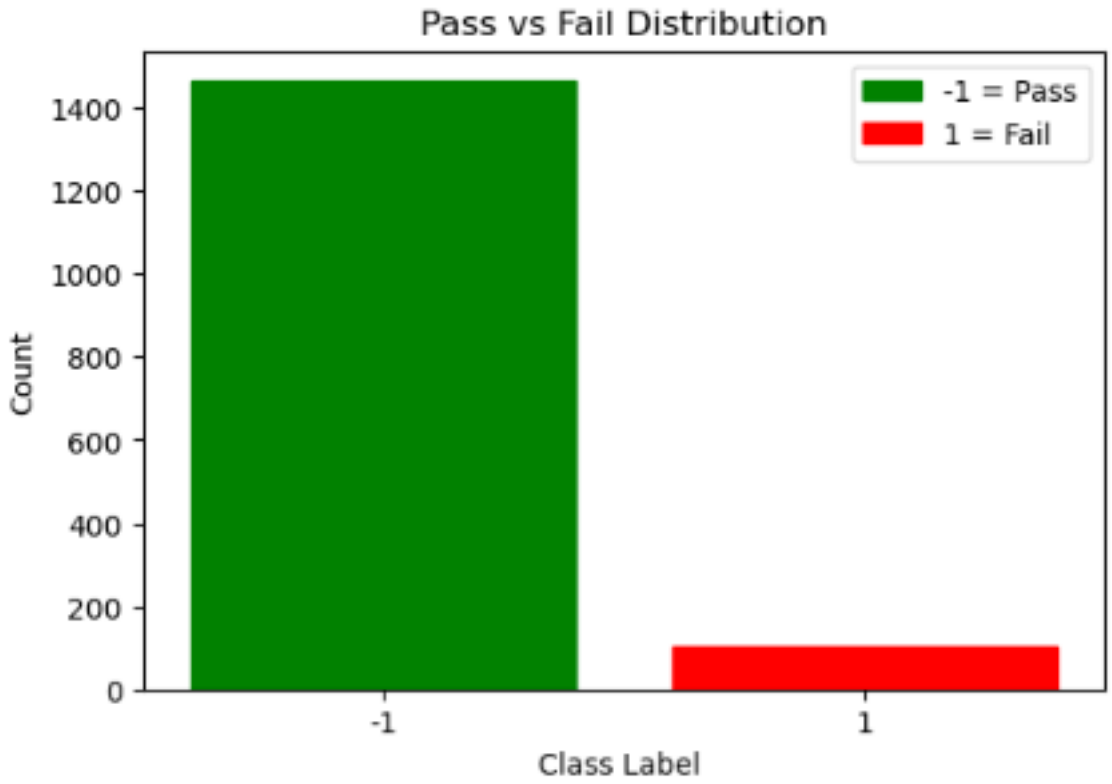
| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| **count** | 1567.00 | 1567.00 | 1567.00 | 1567.00 | 1567.00 |
| **mean** | 0.00 | -0.00 | -0.00 | -0.00 | 0.00 |
| **std** | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| **min** | -3.69 | -4.20 | -4.76 | -3.17 | -0.06 |
| **25%** | -0.65 | -0.54 | -0.66 | -0.71 | -0.06 |
| **50%** | -0.04 | 0.04 | 0.02 | -0.25 | -0.05 |
| **75%** | 0.57 | 0.53 | 0.60 | 0.44 | -0.05 |
| **max** | 4.65 | 4.37 | 3.91 | 5.28 | 19.80 |

Final dataset: (1567, 446)

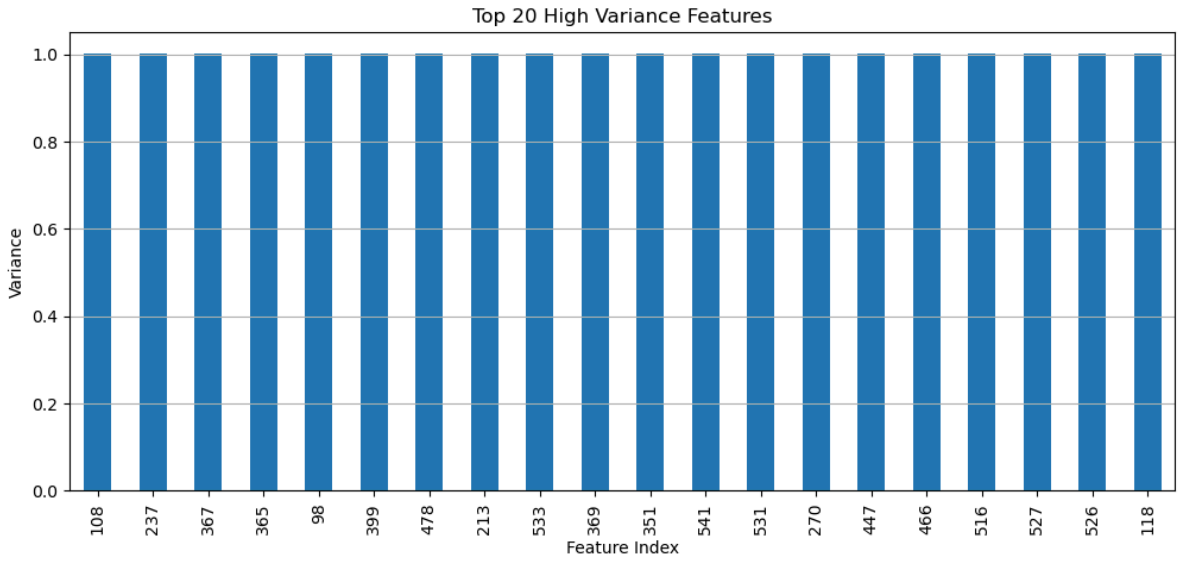


### 3.3 Exploratory Data Analysis

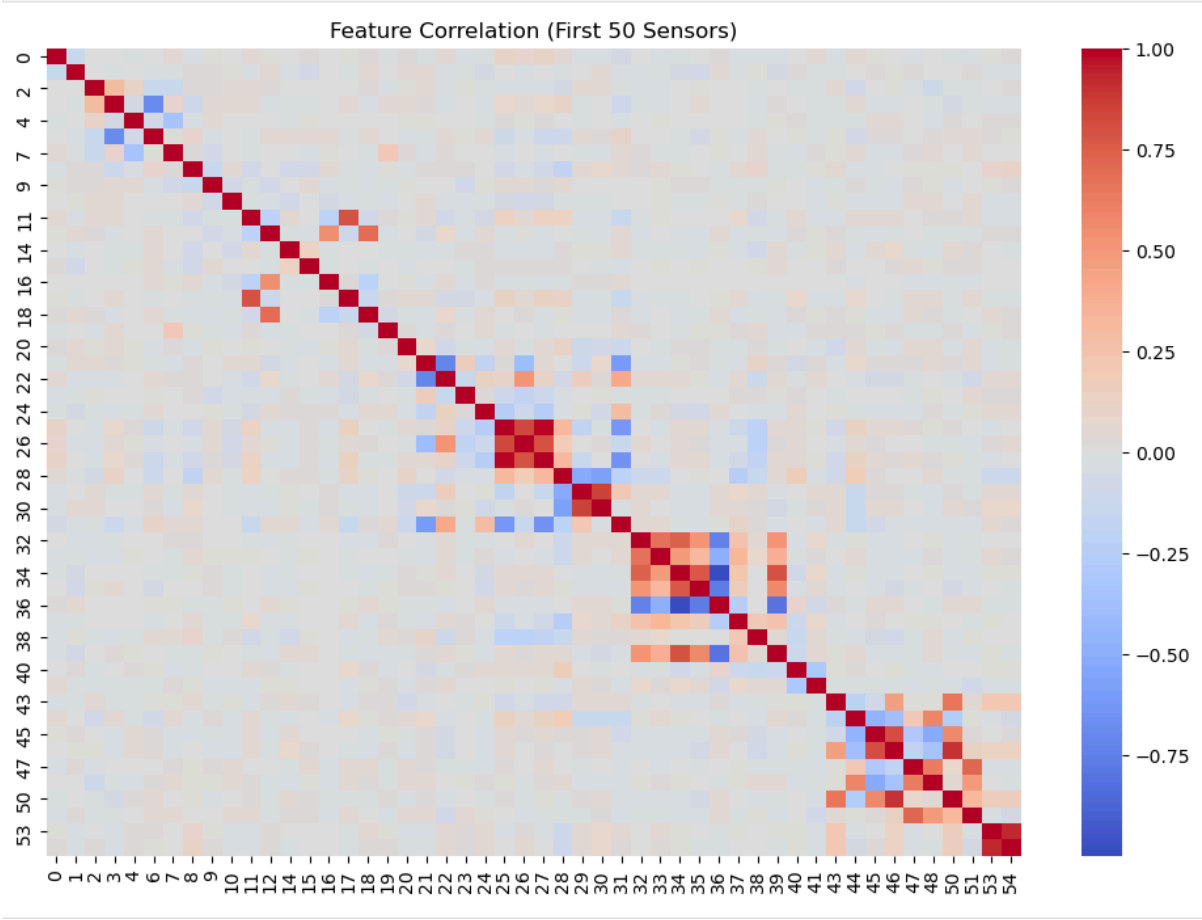
Scaled data set was then used for doing EDA. The imbalance of the classes was checked and plotted, confirming a severe problem with the distribution - 1,463 passing samples (93.4%) and 104 failing ones (6.6%).




The variance of the features post-scaling was analysed to determine which were the most informative features.

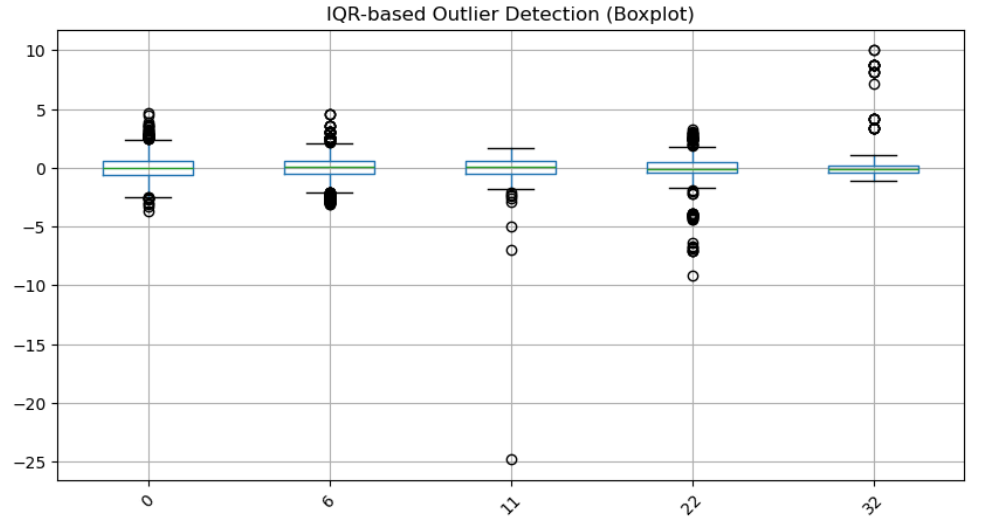


To this end, a correlation heatmap was generated for the first 50 sensor features to investigate relationships between features, revealing multiple sets of sensors that were highly correlated with one another to the extent indicating inter-feature redundancy and warranting PCA-based removal later.

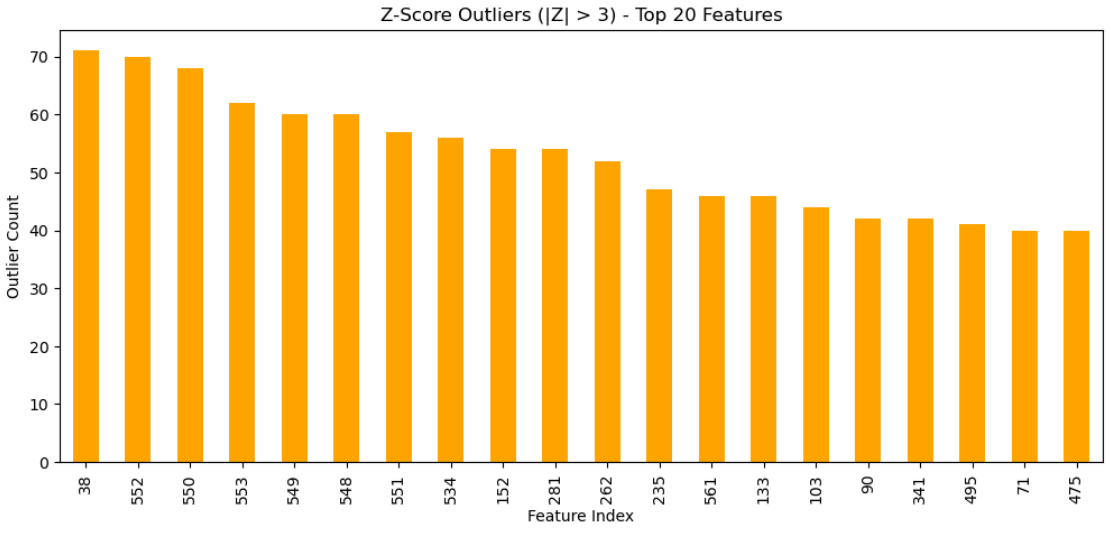


### 3.4 Outlier Analysis

Outlier detection method, Two outlier detection methods were applied including Interquartile Range (IQR) and Z-score analysis. The IQR method detects any data point greater than 1.5 * IQR above Q3 or below Q1. Z-score - Computes Z score on the dataset, flags absolute |Z| value greater than 3 Both approaches were performed over all features and compared the total outlier counts. We had decided to keep all the outliers within our data. In the manufacturing domain, excess readings are most likely to be actual process faults and thus probably holds the highest signal information about product failure. If you did remove them, you would be throwing away exactly the type of signal that the classifier needs to make its decision.



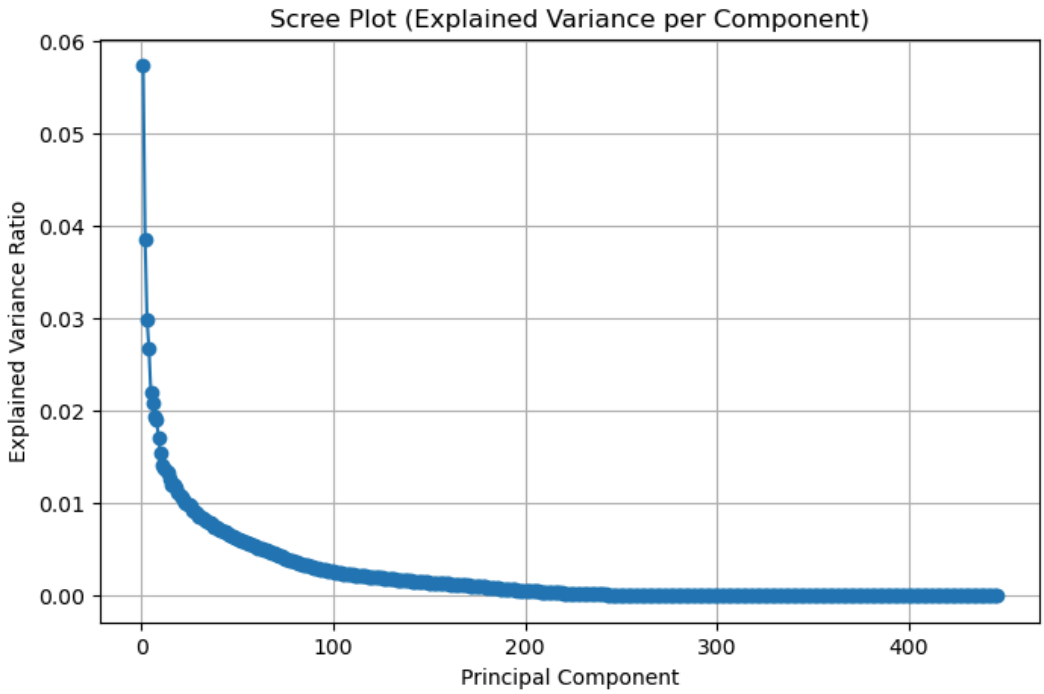

Total IQR Outliers: 26764

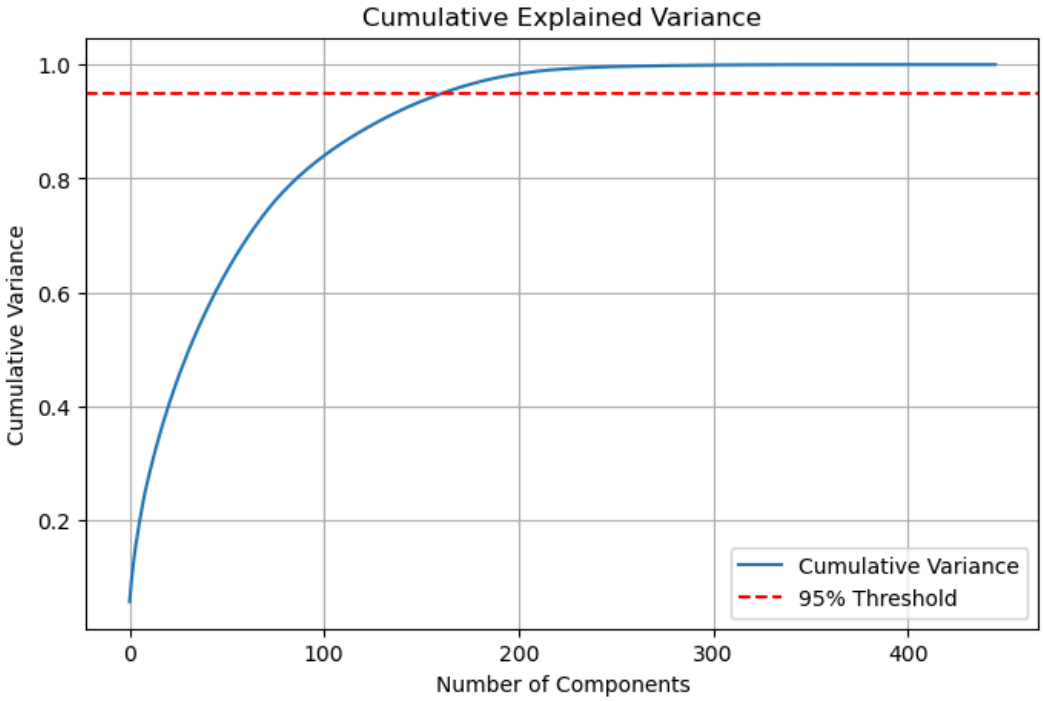



Total Z-Score Outliers: 6198

### 3.5 Dimensionality Reduction (PCA)

Then PCA applied on full scaled dataset To assess how many components to retain, a scree plot and cumulative explained variance curve were generated.



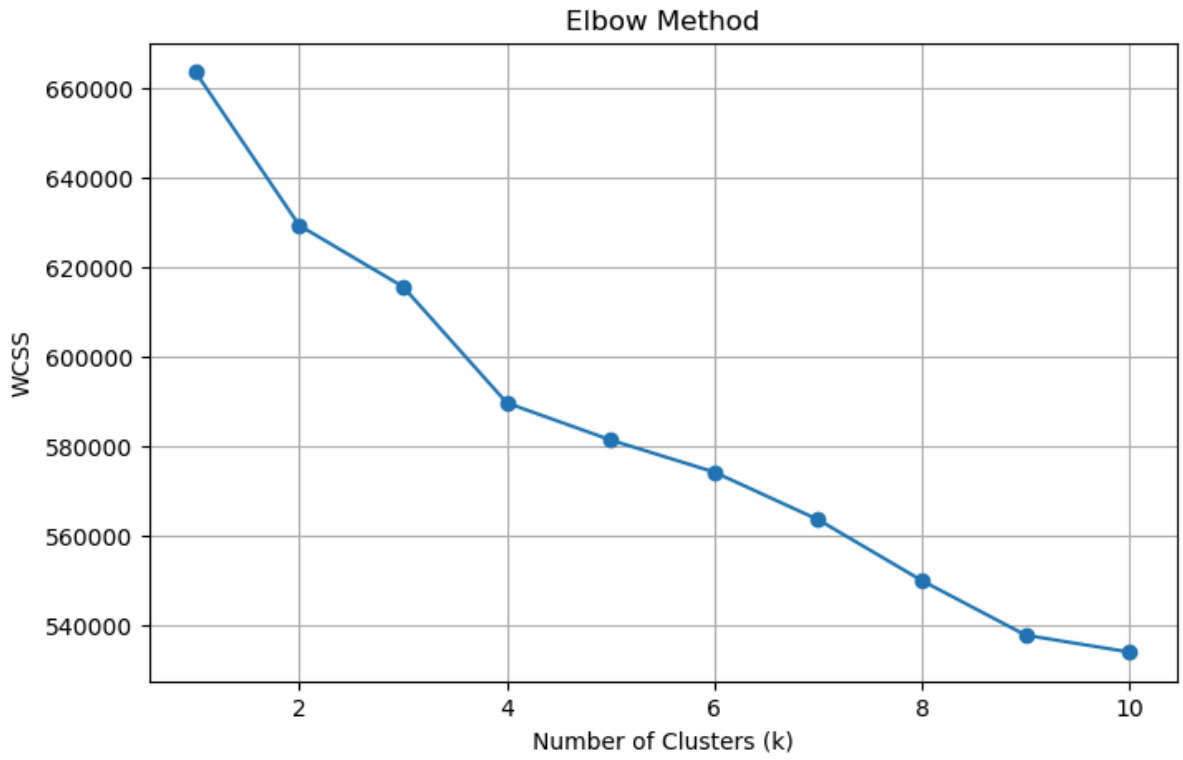
We examined the variance retained at each threshold over six thresholds (70%, 80%, 85%, 90% and 95% and through an aggregated >99% sign threshold).

| Variance Threshold | Feature count |
|---|---|
| 70% | 63 |
| 80% | 87 |
| 85% | 105 |
| 90% | 129 |
| **95% (Selected)** | 162 |
| 99% | 216 |

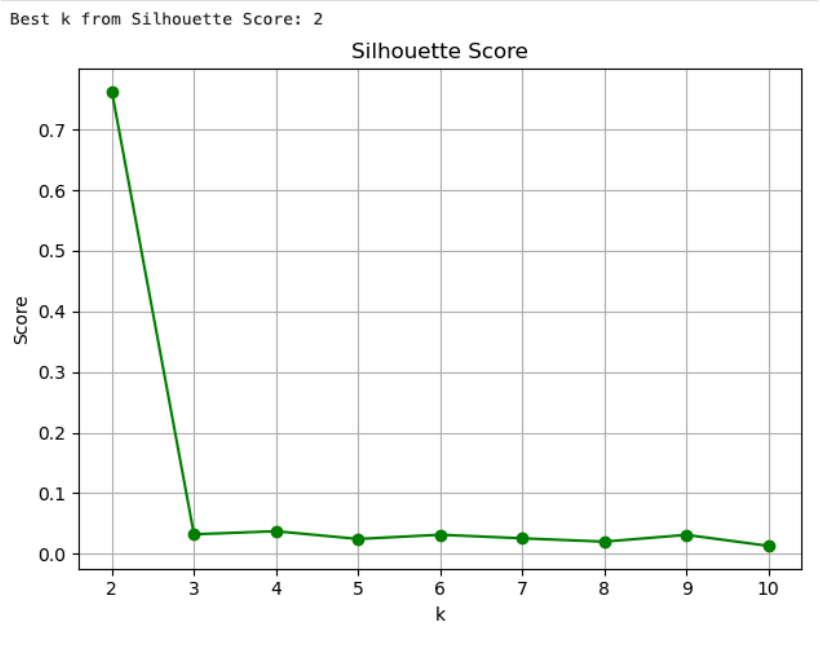
To reduce the number of features retained in them a threshold of 95% was set, where by only the amount of components needed to explain at least 95% of the total variance in the data were retained. This value was selected based on a trade-off between compressing the feature space heavily and maintaining sufficient variance for meaningful classification. All clustering and classification steps were performed on the reduced dataset.

### 3.6 Clustering

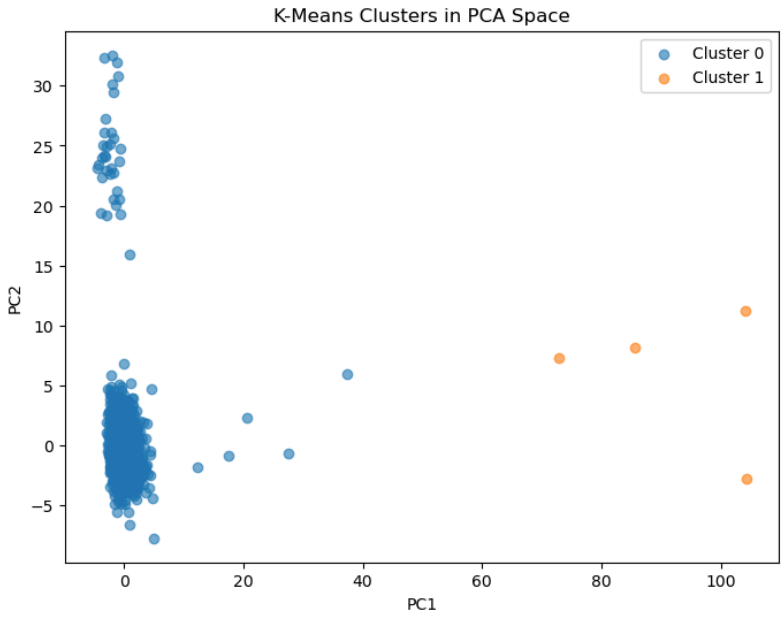
The PCA-reduced dataset was then subjected to K-Means clustering. Two methods were applied to find the number of clusters. The elbow method plots WCSS versus k for 1 to 10.




The silhouette score is calculated for k ranging from 2 to 10, the highest k value provides more compact and very distinct clusters.



Both methods indicated k= 2 was a suitable choice–consistent which the binary nature of the classification problem.




```
Cluster vs True Label:
True Label    -1    1
Cluster
0           1460  103
1              3    1
```

After fitting, clustering assignments were cross tabbed against the known labels (pass/fail) to determine if the clusters captured that pass/fail structure.

### 3.7 Classification

Three classifiers were implemented:

1. k-NN was evaluated using different values of *k* (2, 3, 4, 5 and 6 ) to determine the optimal neighborhood size for classification performance

```
Results for k = 2
Accuracy: 0.9394904458598726
Precision: 0.75
Recall: 0.14285714285714285
F1-score: 0.24

Results for k = 3
Accuracy: 0.9426751592356688
Precision: 0.8
Recall: 0.19047619047619047
F1-score: 0.3076923076923077

Results for k = 4
Accuracy: 0.9363057324840764
Precision: 1.0
Recall: 0.047619047619047616
F1-score: 0.09090909090909091

Results for k = 5
Accuracy: 0.9394904458598726
Precision: 1.0
Recall: 0.09523809523809523
F1-score: 0.17391304347826086

Results for k = 6
Accuracy: 0.9331210191082803
Precision: 0.0
Recall: 0.0
F1-score: 0.0
```

k-Nearest Neighbours (k-NN) with k=5 is selected with best recall, sine we have class imbalance we should focus on recall than accuracy.

2. Decision Tree with default parameters and a fixed random seed

Multiple configurations of the Decision Tree classifier were evaluated, including baseline, depth-tuned, and class-weighted versions. Despite hyperparameter tuning and class balancing, the model consistently failed to achieve meaningful recall on the minority class.

```
Baseline Decision Tree
Decision Tree Accuracy: 0.8821656050955414
Precision: 0.0
Recall: 0.0
F1-score: 0.0


Final Model (Balanced Decision Tree)
Accuracy: 0.7547770700636943
Precision: 0.0
Recall: 0.0
F1-score: 0.0
```

3. SVM classifier (specifically, an RBF kernel) SVM with RBF kernel was used as SVM is able to model non-linear decision boundaries, which is expected considering the complexity of sensors data. A stratified sampling with an 80/20 split was made from the data to create a training set and test one, preserving class ratio in both splits. Models were trained in a first step on the original imbalanced training set and then, following SMOTE application to the training set only. SMOTE was only applied to the training set to prevent leaking information about the test set.

### 3.8 SMOTE

Since we had class imbalance SMOTE has been used to enrich the minority class

```
Before SMOTE: (1253, 162) 0
-1    1170
 1      83
Name: count, dtype: int64
After SMOTE: (2340, 162) 0
-1    1170
 1    1170
Name: count, dtype: int64
```

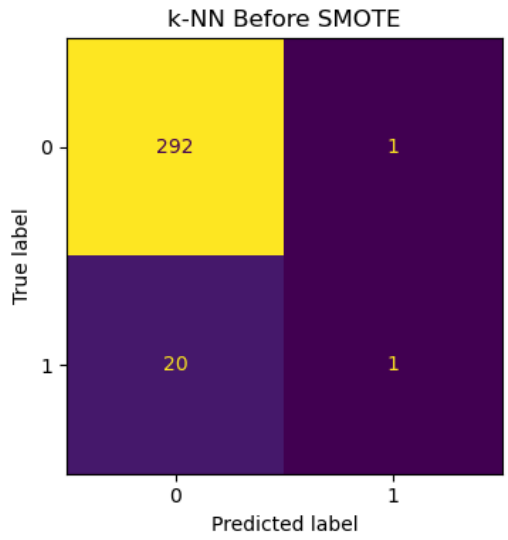
All the classifier models are retrained with SMOTE, and accuracy is compared

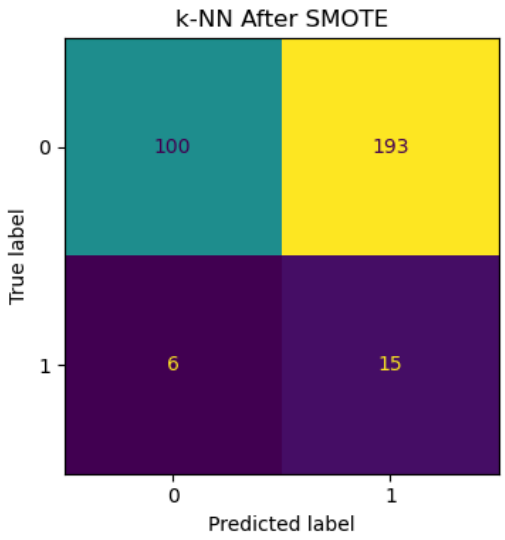
| | Model | Accuracy_Before | Accuracy_After_SMOTE |
|---|---|---|---|
| 0 | k-NN | 0.939490 | 0.359873 |
| 1 | Decision Tree | 0.754777 | 0.789809 |
| 2 | SVM | 0.933121 | 0.929936 |

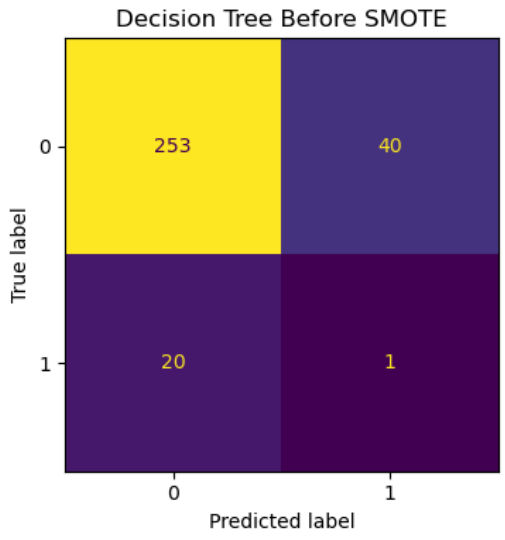
The performance degradation of k-NN after applying SMOTE indicates that synthetic sampling can negatively affect distance-based classifiers. Since k-NN relies heavily on local neighborhood structure, the introduction of synthetic samples alters the natural data distribution, leading to less meaningful nearest-neighbor relationships. In contrast, SVM remains robust due to its margin-based decision boundary, and Decision Tree shows moderate stability.

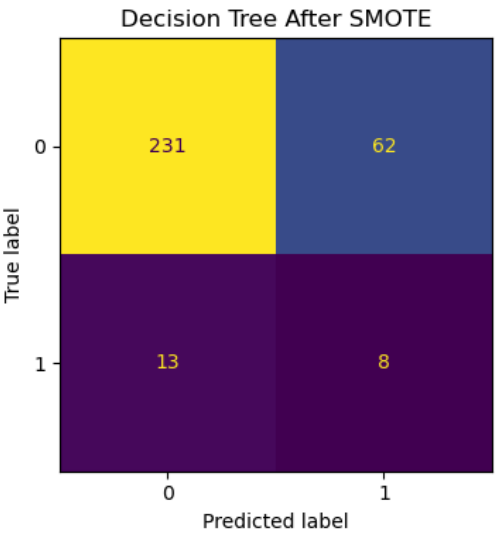
### 3.9 Model Evaluation

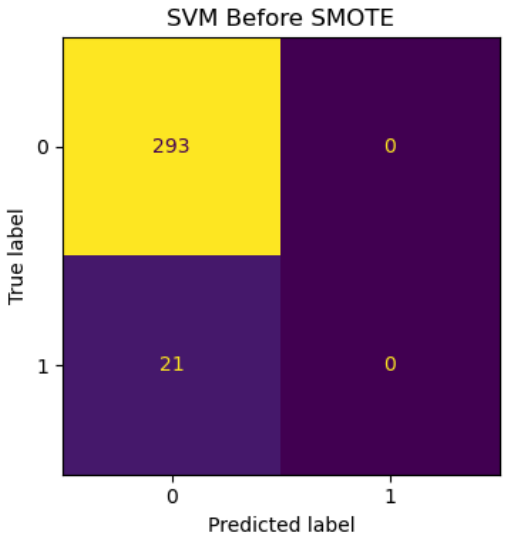
Before and after SMOTE, each model was evaluated using accuracy, precision, recall and F1 score. The confusion matrices were generated for the six variants of the models. A classifier which always predicts pass would achieve 93.4% accuracy, so for class imbalanced datasets mere accuracy is a bad metric. Through the lens of having missed a failing unit being at least as costly in reality as misclassifying a healthy one, recall for the fail class was therefore considered the primary metric.SMOTE improves minority class detection but affects models differently depending on their learning mechanism.

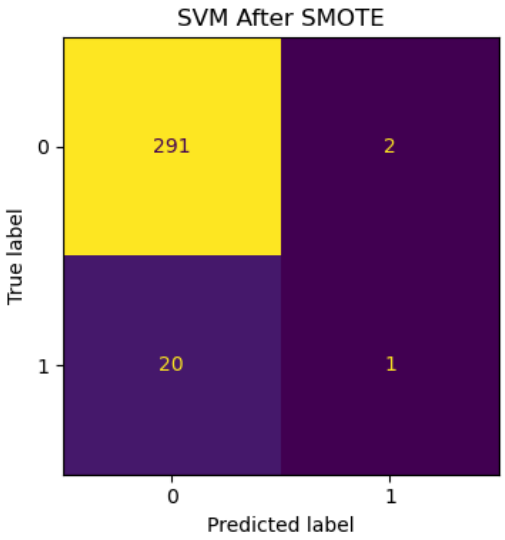

k-NN Before SMOTE
292
1
20
1
0
1
True label
0
1
Predicted label


k-NN After SMOTE
100
193
6
15
0
1
True label
0
1
Predicted label


Decision Tree Before SMOTE
253
40
20
1
0
1
True label
0
1
Predicted label


Decision Tree After SMOTE
231
62
13
8
0
1
True label
0
1
Predicted label


SVM Before SMOTE
293
0
21
0
0
1
True label
0
1
Predicted label


SVM After SMOTE
291
2
20
1
0
1
True label
0
1
Predicted label

# 4. Experiments and Results

## 4.1 Preprocessing Outcomes

After removing features with 50% or more missing values, the dataset was reduced from 590 features to 562.

116 Zero-variance features were then removed, and the final dataset after imputation and variance filtering was used for scaling. The total number of missing values dropped to zero after median imputation. The before and after scaling distributions confirmed that standardisation shifted the feature distributions to zero mean and unit variance as expected.

```
Final dataset: (1567, 446)
```

## 4.2 EDA Findings

The class distribution confirmed 1,463 passing samples versus 104 failing, a ratio of roughly 14:1. The top variance features post-scaling were identified and plotted. The correlation heatmap revealed notable clusters of correlated sensor features among the first 50, consistent with the expectation that many sensors in a production line measure related aspects of the same underlying process. These correlations provide further motivation for applying PCA.

## 4.3 Outlier Detection

The IQR method detected a substantially higher number of outlier data points than the Z-score method. This is expected, as IQR is more sensitive to deviations from the central distribution, while Z-score with a threshold of 3 only captures extreme values. The top 20 features by Z-score outlier count were plotted, showing that certain sensor channels account for a disproportionate share of extreme readings. As discussed in the methodology, all outliers were retained.

```
Total IQR Outliers: 26764
```

```
Total Z-Score Outliers: 6198
```

### 4.4 PCA Results

A Common pattern observed in high dimensional sensor datasets is rapid decline in explained variance per component, which can be seen from the scree plot. The cumulative variance curve crossed the classic 95% limit at far less than the size of the original feature set, indicating a high degree of dataset redundancy. The component counts per threshold are summarised as follows:

| Variance Threshold | Feature count |
|---|---|
| 70% | 63 |
| 80% | 87 |
| 85% | 105 |
| 90% | 129 |
| **95% (Selected)** | 162 |
| 99% | 216 |

The 95% threshold was selected and the dataset reduced to the corresponding number of components. This reduced dataset was used for all subsequent steps.

### 4.5 Clustering Results

The elbow plot for K-means indicated that the reduction in WCSS beyond k=2 was diminishing returns, and the maximum silhouette score was at k=2 - supporting this choice. Fitting with k=2 resulted in the following clusters distribution and true labels cross-tabulation:

```
Cluster vs True Label:
True Label    -1    1
Cluster
0           1460  103
1              3    1
```

The K-Means did not perform well in separating pass/fail units, as can be seen from the cross tab. Cluster 0 contains nearly all of both classes, and Cluster 1 is a small group with only four samples. There are two reasons why such a result is anticipated.

- K-Means in an unsupervised method which does not have access to the class labels and it groups by feature similarity and not outcome.
- The extreme imbalance of classes will cause the centroid of any meaningful cluster to be dominated by passing units. These results demonstrate the requirement for a supervised method and motivate the application of SMOTE to mitigate the imbalance.

### 4.6 Classification Results

The three classifiers were trained and tested with and without the SMOTE oversampling. Accuracies, precisions, recalls and F1 scores are added for reference.

BEFORE SMOTE

| Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| **k-NN** | 0.9395 | 1.0000 | 0.0952 | 0.1739 |
| **Decision Tree** | 0.8217 | 0.0513 | 0.0952 | 0.0667 |
| **SVM** | 0.9331 | 0.0000 | 0.0000 | 0.0000 |

AFTER SMOTE

| Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| **k-NN** | 0.3471 | 0.0701 | 0.7143 | 0.1277 |
| **Decision Tree** | 0.7962 | 0.1228 | 0.3333 | 0.1795 |
| **SVM** | 0.9299 | 0.3333 | 0.0476 | 0.0833 |

If we do not apply any method, using k-NN and SVM predict the majority class so we have very high accuracy but recall on failing unit was almost zero. The Decision Tree performed better on recall due to separating decision regions that were more specific but worse on accuracy. All the three models improved considerably in fail recall after SMOTE. The best balance of precision and recall was achieved with Decision Tree applied using SMOTE, scoring the highest F1 value of 0.1795, while SVM despite gaining some recall still scored the lowest F1 of 0.0183.

# 5. Discussion

The results across all stages of this project tell a coherent and somewhat expected story about what happens when standard data mining methods are applied to an imbalanced, high-dimensional industrial dataset.

The preprocessing decisions governed how well the data could be consumed at later stages. The more than 50% missing values criterion for feature removal was easy to justify, but this threshold is also somewhat arbitrary. A more extensive approach would be to test several thresholds and evaluate their impact on downstream classifier performance. In this case, median imputation was a reasonable choice due to sensor readings being skewed. However, more sophisticated methods such as k-NN imputation could potentially recover additional signal from the non-missing values.

The EDA confirmed what the dataset description implies. The class imbalance is so severe that any model evaluated using accuracy alone will appear to perform adequately while failing significantly at its actual task. This is a frequent pitfall in industrial fault detection, which is why recall and F1-score are much more relevant metrics in this scenario.

Outlier analyis highlighted that this type of dataset inherently contains a tension. Extreme sensor readings may represent noise, but they may also indicate early signs of manufacturing faults. Removing them could make the decision boundary for passing units smoother but may reduce the model's ability to detect failures. The more prudent and defensible choice is to retain them, as the primary objective is fault detection.

This is where PCA becomes relevant, and the results support its effectiveness. The scree plot showed the expected steep initial decline followed by a long tail, indicating that most variance is captured by a small number of components while the remaining components largely represent noise. The 95% variance threshold reduced the feature space significantly while preserving most of the signal. However, this introduces an important trade-off. PCA reduces interpretability, meaning individual sensor contributions to failure prediction can no longer be

directly identified. This may be acceptable in some predictive settings, but in industrial applications, interpretability is often desirable for actionable insights.

The clustering results were also informative in showing the limitations of unsupervised learning in this context. The cross-tabulation indicated that one cluster contained almost all samples from both classes, while the second cluster consisted of only a few outliers. This suggests that there is no clear unsupervised structure in the feature space that aligns with pass/fail labels. This is intuitive, as failed and passing units are likely distinguished by subtle, nonlinear interactions among sensor readings rather than simple geometric separation. This directly justifies the need for supervised learning approaches.

The prior pattern was evident before applying SMOTE across all classifiers, with high accuracy but near-zero recall for the failure class. This reflects the standard majority-class dominance problem. After applying SMOTE, all models improved in terms of minority class detection, but the Decision Tree with SMOTE provided the best overall balance between precision and recall (F1-score), making it the most effective model in this study.

This aligns partially with expectations from the literature. SVMs are well-suited to high-dimensional feature spaces due to their maximum-margin formulation, which can generalize effectively under limited data conditions. However, in this dataset, their performance is limited by class imbalance sensitivity and reduced ability to improve recall after SMOTE. In contrast, Decision Trees are more sensitive to data distribution changes but benefit more from oversampling techniques, which improves their balance between precision and recall in this case. k-NN depends heavily on local density structure, and while SMOTE improves class balance, the transformed PCA space may still not produce sufficiently meaningful neighborhood structure for optimal performance.

At this stage, it is important to acknowledge certain limitations of this work. First, only a single 80/20 train-test split was used. A more robust evaluation could be achieved using cross-validation, particularly given the small number of failure samples. Second, although basic hyperparameter exploration was performed (such as selecting k for k-NN and max_depth for

the Decision Tree), no systematic optimization methods such as grid search or cross-validation-based tuning were applied. Therefore, the reported models may not represent globally optimal configurations. Fine-tuning these parameters could potentially lead to improved performance. Finally, PCA reduces interpretability despite improving computational efficiency. In a real semiconductor manufacturing environment, engineers would typically require explanations of which specific sensors contribute most to failure prediction. PCA does not provide this directly, whereas tree-based models or SHAP-based interpretations would be more suitable for operational decision-making.

# 6. Conclusion

This project applied a complete data mining pipeline to the SECOM semiconductor manufacturing dataset. Starting from a raw, messy, high-dimensional sensor dataset with severe class imbalance, we progressively cleaned the data, performed EDA, reduced its dimensionality, and applied both unsupervised and supervised learning methods.

The key findings are -

- Preprocessing removed a substantial number of uninformative features and brought the data into a form suitable for machine learning.
- EDA confirmed the class imbalance and revealed inter-sensor correlations that justified dimensionality reduction.
- Outlier analysis showed that extreme readings are present across many features and were retained given their potential relevance to fault detection.
- PCA reduced the feature space down to just a few components while keeping 95% of variance. Even K-Means clustering with the ideal choice of k=2 was unable to recover the pass/fail structure, demonstrating how difficult unsupervised methods can be on heavily imbalanced data. Of the classifiers, Decision Tree with SMOTE showed best results, obtaining meaningful recall on the minority (failure) class after oversampling.

Future work can expand upon cross-validation, attempt hyperparameter optimisation, explore alternative imputation strategies as well as interpretable methods of feature selection that maintain our ability to identify specific faulty sensor channels. You could also try ensemble methods like Random Forest or Gradient Boosting which are better suited for a class imbalance and high dimensional data scenario.

# 7. References

**[1]** N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. https://doi.org/10.1613/jair.953

**[2]** C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995. https://doi.org/10.1007/BF00994018

**[3]** I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002. https://link.springer.com/book/9780387954424

**[4]** M. McCann and A. Johnston, "SECOM Dataset," UCI Machine Learning Repository, 2008. https://archive.ics.uci.edu/dataset/179/secom

**[5]** F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

# Appendix

The following Python code was used to implement the data mining pipeline described in this report.

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.patches as mpatches

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

from sklearn.model_selection import train_test_split

from sklearn.cluster import KMeans

from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    silhouette_score
)

from imblearn.over_sampling import SMOTE

X = pd.read_csv(
    "secom.data",
    sep=r"\s+",
```

```
    header=None
)

labels = pd.read_csv(
    "secom_labels.data",
    sep=r"\s+",
    header=None
)

# First column contains labels
y = labels[0]
```

## Data Understanding

```
print("Dataset Shape:")
print("Samples:", X.shape[0])
print("Features:", X.shape[1])

print("\nTarget Labels Distribution:")
print(y.value_counts())

# Percentage of missing values per feature
missing_percentage = X.isnull().mean() * 100

# Top 50 features with highest missing values
top_missing = missing_percentage.sort_values(ascending=False).head(50)

plt.figure(figsize=(14,5))

top_missing.plot(kind='bar')

plt.title("Top 50 Features with Highest Missing Values")
plt.xlabel("Feature Index")
plt.ylabel("Missing Percentage")

plt.grid(axis='y')
plt.show()

print("Features with more than 50% missing values:")
```

```
print((missing_percentage > 50).sum())


threshold = 0.5
removed_features = (X.isnull().mean() >= threshold).sum()
print("\nNumber of features removed:", removed_features)
X = X.loc[:, X.isnull().mean() < threshold]
print("Shape after removing high missing features:", X.shape)


print("\nMissing values before imputation:")
print(X.isnull().sum().sum())


imputation = SimpleImputer(strategy='median')
X_imputed = imputation.fit_transform(X)
print("\nShape after imputation:", X_imputed.shape)
X_imputed_df = pd.DataFrame(X_imputed)
print("Remaining missing values after median imputation:")
print(X_imputed_df.isnull().sum().sum())


#Variance analysis
variance = X_imputed_df.var()


print("\nSummary of feature variance:")
print(variance.describe())


# Top 20 highest variance features
top_variance = variance.sort_values(ascending=False).head(20)
plt.figure(figsize=(12,5))
top_variance.plot(kind='bar')
plt.title("Top 20 High Variance Features")
plt.xlabel("Feature Index")
plt.ylabel("Variance")
plt.grid(axis='y')
plt.show()


# Count zero variance features
zero_variance_count = (variance == 0).sum()
print("Number of zero variance features removed:",
```

```
      zero_variance_count)

# Remove zero variance features
X_final = X_imputed_df.loc[:, variance > 0]
print("Final dataset shape after variance filtering:",
      X_final.shape)

# Before scaling
print("BEFORE SCALING (first 5 features):")
display(X_final.iloc[:, :5].describe().round(2))

# Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_final)

# After scaling (convert to DataFrame for inspection)
X_scaled_df = pd.DataFrame(X_scaled, columns=X_final.columns)

print("\nAFTER SCALING (first 5 features):")
display(X_scaled_df.iloc[:, :5].describe().round(2))

print("\nFinal dataset:", X_scaled_df.shape)

# BEFORE SCALING
plt.figure(figsize=(10,5))

plt.hist(X_final.iloc[:, 0], bins=30, alpha=0.6, label="Feature 0 (Before)")
plt.hist(X_final.iloc[:, 1], bins=30, alpha=0.6, label="Feature 1 (Before)")

plt.title("Feature Distribution BEFORE Scaling")
plt.legend()
plt.show()

# AFTER SCALING
plt.figure(figsize=(10,5))

plt.hist(X_scaled_df.iloc[:, 0], bins=30, alpha=0.6, label="Feature 0 (After)")
plt.hist(X_scaled_df.iloc[:, 1], bins=30, alpha=0.6, label="Feature 1 (After)")
```

```
plt.title("Feature Distribution AFTER Standardization")
plt.legend()
plt.show()
```

## EDA

```
X_df = pd.DataFrame(X_scaled, columns=X_final.columns)


X_df.describe().round(2)


plt.figure(figsize=(6,4))


ax = sns.countplot(x=y)


ax.patches[0].set_color("green")  # -1 = Pass
ax.patches[1].set_color("red")    # 1 = Fail


plt.title("Pass vs Fail Distribution")
plt.xlabel("Class Label ")
plt.ylabel("Count")


pass_patch = mpatches.Patch(color="green", label='-1 = Pass')
fail_patch = mpatches.Patch(color="red", label='1 = Fail')


plt.legend(handles=[pass_patch, fail_patch])


plt.show()


print("Class distribution:")
print(y.value_counts())


fail_rate = (y.value_counts(normalize=True) * 100)


print("\nClass percentage distribution:")
print(fail_rate.round(2))


variance = X_df.var().sort_values(ascending=False)
```

```
print("Top 100 high variance features:")
display(variance.head(100))


# Top 20 high variance features
top_variance = variance.head(20)

plt.figure(figsize=(12,5))

top_variance.plot(kind='bar')

plt.title("Top 20 High Variance Features")
plt.xlabel("Feature Index")
plt.ylabel("Variance")

plt.grid(axis='y')
plt.show()


plt.figure(figsize=(12,8))

corr = X_df.iloc[:, :50].corr()

sns.heatmap(
    corr,
    cmap="coolwarm",
    center=0,
    linewidths=0
)

plt.title("Feature Correlation (First 50 Sensors)")
plt.show()
```

## Outlier

```
X_df = pd.DataFrame(X_scaled, columns=X_final.columns)


# IQR calculation
Q1 = X_df.quantile(0.25)
Q3 = X_df.quantile(0.75)
IQR = Q3 - Q1
```

```
iqr_outliers = ((X_df < (Q1 - 1.5 * IQR)) | (X_df > (Q3 + 1.5 * IQR)))

iqr_count = iqr_outliers.sum().sum()

# Boxplot (Random sample features)
sample_cols = X_df.iloc[:, [0, 5, 10, 20, 30]]

plt.figure(figsize=(10,5))
sample_cols.boxplot()

plt.title("IQR-based Outlier Detection (Boxplot)")
plt.xticks(rotation=45)

plt.show()

print("Total IQR Outliers:", iqr_count)

# Z-scores
z_scores = (X_df - X_df.mean()) / X_df.std()

z_outliers = np.abs(z_scores) > 3

zscore_count = z_outliers.sum().sum()

z_feature = z_outliers.sum().sort_values(ascending=False).head(20)

# Plot
plt.figure(figsize=(12,5))

z_feature.plot(kind='bar', color='orange')

plt.title("Z-Score Outliers (|Z| > 3) - Top 20 Features")
plt.xlabel("Feature Index")
plt.ylabel("Outlier Count")

plt.show()
```

```
print("Total Z-Score Outliers:", zscore_count)


comparison = pd.DataFrame({
    "Method": ["IQR", "Z-Score"],
    "Outlier Count": [iqr_count, zscore_count]
})

print("Outlier Detection Comparison:")
display(comparison)
```

## PCA

```
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

explained_variance = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

print("First 10 component variance ratios:")
print(explained_variance[:10])

# Scree plot
plt.figure(figsize=(8,5))

plt.plot(range(1, len(explained_variance)+1),
         explained_variance, marker='o')

plt.title("Scree Plot (Explained Variance per Component)")
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")

plt.grid()
plt.show()

plt.figure(figsize=(8,5))

plt.plot(cumulative_variance, label="Cumulative Variance")
plt.axhline(y=0.95, color='red', linestyle='--', label="95% Threshold")
```

```
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance")
plt.title("Cumulative Explained Variance")

plt.legend()
plt.grid()
plt.show()

# components for 90% ,95% and 99%
n_70 = np.argmax(cumulative_variance >= 0.70) + 1
n_80 = np.argmax(cumulative_variance >= 0.80) + 1
n_85 = np.argmax(cumulative_variance >= 0.85) + 1
n_90 = np.argmax(cumulative_variance >= 0.90) + 1
n_95 = np.argmax(cumulative_variance >= 0.95) + 1
n_99 = np.argmax(cumulative_variance >= 0.99) + 1

print("Components needed for:")
print("70% variance :", n_70)
print("80% variance :", n_80)
print("85% variance :", n_85)
print("90% variance :", n_90)
print("95% variance :", n_95)
print("99% variance :", n_99)

pca_final = PCA(n_components=n_95)
X_reduced = pca_final.fit_transform(X_scaled)
print("Dataset shape after PCA:", X_reduced.shape)
```

## K - MEANS

```
wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_reduced)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(range(1, 11), wcss, marker='o')
```

```
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS")

plt.grid()
plt.show()


sil_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(X_reduced)

    score = silhouette_score(X_reduced, cluster_labels)
    sil_scores.append(score)

best_k = range(2, 11)[np.argmax(sil_scores)]
print("Best k from Silhouette Score:", best_k)

plt.plot(range(2, 11), sil_scores, marker='o', color='green', label="Silhouette
Score")
plt.title("Silhouette Score")
plt.xlabel("k")
plt.ylabel("Score")
plt.grid()

plt.tight_layout()
plt.show()


k = 2  # chosen based on elbow and silhoutte method

kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_reduced)

print("Number of clusters selected:", k)
print("Cluster labels:", np.unique(clusters))
```

```
cluster_counts = pd.Series(clusters).value_counts().sort_index()


print("Cluster distribution:")
print(cluster_counts)


plt.figure(figsize=(8,6))


for label in np.unique(clusters):
    plt.scatter(
        X_reduced[clusters == label, 0],
        X_reduced[clusters == label, 1],
        label=f"Cluster {label}",
        alpha=0.6
    )


plt.title("K-Means Clusters in PCA Space")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.show()


print("\nCluster vs True Label:")
print(pd.crosstab(clusters, y, rownames=['Cluster'], colnames=['True Label']))
```

## Classifiers

```
X_train, X_test, y_train, y_test = train_test_split(
    X_reduced, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)


print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

## KNN

```
#k - Comparison
k_values = [2, 3, 4, 5,6]
```

```
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    y_pred_knn = knn.predict(X_test)

    print(f"\nResults for k = {k}")
    print("Accuracy:", accuracy_score(y_test, y_pred_knn))
    print("Precision:", precision_score(y_test, y_pred_knn, zero_division=0))
    print("Recall:", recall_score(y_test, y_pred_knn, zero_division=0))
    print("F1-score:", f1_score(y_test, y_pred_knn, zero_division=0))

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

knn_acc = accuracy_score(y_test, y_pred_knn)

print("k-NN Accuracy:", knn_acc)
```

## Decision Tree

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

dt_acc = accuracy_score(y_test, y_pred_dt)

print("Baseline Decision Tree")
print("Decision Tree Accuracy:", dt_acc)
print("Precision:", precision_score(y_test, y_pred_dt, zero_division=0))
print("Recall:", recall_score(y_test, y_pred_dt, zero_division=0))
print("F1-score:", f1_score(y_test, y_pred_dt, zero_division=0))

depth_values = [3, 5, 10, 15, 20, None]
```

```
for depth in depth_values:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)

    y_pred = dt.predict(X_test)

    print(f"\nDepth = {depth}")
    print("F1:", f1_score(y_test, y_pred, zero_division=0))

dt_balanced = DecisionTreeClassifier(
    max_depth=10,
    class_weight="balanced",
    random_state=42
)

dt_balanced.fit(X_train, y_train)
y_pred_dt = dt_balanced.predict(X_test)

dt_balanced_acc = accuracy_score(y_test, y_pred_dt)

print("Final Model (Balanced Decision Tree)")
print("Accuracy:", dt_balanced_acc)
print("Precision:", precision_score(y_test, y_pred_dt, zero_division=0))
print("Recall:", recall_score(y_test, y_pred_dt, zero_division=0))
print("F1-score:", f1_score(y_test, y_pred_dt, zero_division=0))
```

## SVM Classifier

```
#for complex boundaries we are using rbf kernel

svm = SVC(kernel='rbf', random_state=42)
svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)

svm_acc = accuracy_score(y_test, y_pred_svm)

print("SVM Accuracy:", svm_acc)
```

```
results = pd.DataFrame({
    "Model": ["k-NN", "Balanced Decision Tree", "SVM"],
    "Accuracy": [knn_acc, dt_balanced_acc, svm_acc]
})

display(results)

best_model = results.loc[results["Accuracy"].idxmax()]

print("Best Model:", best_model["Model"])
print("Best Accuracy:", best_model["Accuracy"])
```

## SMOTE

```
smote = SMOTE(random_state=42)

X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print("Before SMOTE:", X_train.shape, y_train.value_counts())
print("After SMOTE:", X_train_sm.shape, y_train_sm.value_counts())

# Retraining models
# k-NN
knn_sm = KNeighborsClassifier(n_neighbors=5)
knn_sm.fit(X_train_sm, y_train_sm)
y_pred_knn_sm = knn_sm.predict(X_test)

# Decision Tree
dt_sm = DecisionTreeClassifier(
    max_depth=10,
    class_weight="balanced",
    random_state=42
)
dt_sm.fit(X_train_sm, y_train_sm)
y_pred_dt_sm = dt_sm.predict(X_test)

# SVM
svm_sm = SVC(kernel='rbf', random_state=42)
svm_sm.fit(X_train_sm, y_train_sm)
```

```
y_pred_svm_sm = svm_sm.predict(X_test)


comparison = pd.DataFrame({
    "Model": ["k-NN", "Decision Tree", "SVM"],

    "Accuracy_Before": [
        knn_acc, dt_balanced_acc, svm_acc
    ],

    "Accuracy_After_SMOTE": [
        accuracy_score(y_test, y_pred_knn_sm),
        accuracy_score(y_test, y_pred_dt_sm),
        accuracy_score(y_test, y_pred_svm_sm)
    ]
})

display(comparison)
```

## Model Evaluation

```
# K-NN
cm_before = confusion_matrix(y_test, y_pred_knn)
cm_after = confusion_matrix(y_test, y_pred_knn_sm)

fig, axes = plt.subplots(1, 2, figsize=(10,4))

# Before SMOTE
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm_before)
disp1.plot(ax=axes[0], colorbar=False)
axes[0].set_title("k-NN Before SMOTE")

# After SMOTE
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_after)
disp2.plot(ax=axes[1], colorbar=False)
axes[1].set_title("k-NN After SMOTE")

plt.tight_layout()
plt.show()
```

```
# Decision Tree
cm_dt_before = confusion_matrix(y_test, y_pred_dt)
cm_dt_after = confusion_matrix(y_test, y_pred_dt_sm)

fig, axes = plt.subplots(1, 2, figsize=(10,4))

# Before SMOTE
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm_dt_before)
disp1.plot(ax=axes[0], colorbar=False)
axes[0].set_title("Decision Tree Before SMOTE")

# After SMOTE
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_dt_after)
disp2.plot(ax=axes[1], colorbar=False)
axes[1].set_title("Decision Tree After SMOTE")

plt.tight_layout()
plt.show()

# SVM
cm_svm_before = confusion_matrix(y_test, y_pred_svm)
cm_svm_after = confusion_matrix(y_test, y_pred_svm_sm)

fig, axes = plt.subplots(1, 2, figsize=(10,4))

# Before SMOTE
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm_svm_before)
disp1.plot(ax=axes[0], colorbar=False)
axes[0].set_title("SVM Before SMOTE")

# After SMOTE
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_svm_after)
disp2.plot(ax=axes[1], colorbar=False)
axes[1].set_title("SVM After SMOTE")

plt.tight_layout()
plt.show()
```

```
models = [
    ("k-NN", y_pred_knn, y_pred_knn_sm),
    ("Decision Tree", y_pred_dt, y_pred_dt_sm),
    ("SVM", y_pred_svm, y_pred_svm_sm)
]

before_results = []
after_results = []

for name, y_before, y_after in models:

    # BEFORE SMOTE
    before_results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_before),
        "Precision": precision_score(y_test, y_before, zero_division=0),
        "Recall": recall_score(y_test, y_before, zero_division=0),
        "F1 Score": f1_score(y_test, y_before, zero_division=0)
    })

    # AFTER SMOTE
    after_results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_after),
        "Precision": precision_score(y_test, y_after, zero_division=0),
        "Recall": recall_score(y_test, y_after, zero_division=0),
        "F1 Score": f1_score(y_test, y_after, zero_division=0)
    })

df_before = pd.DataFrame(before_results).set_index("Model")
df_after = pd.DataFrame(after_results).set_index("Model")

print("BEFORE SMOTE")
display(df_before.round(4))

print("\nAFTER SMOTE")
display(df_after.round(4))
```